Regression Investigation Report

Date: 2026-06-29
Status: Behavioral bisect in progress
Ref: Direction for Regression Investigation (1).pdf

Summary of Findings

✅ THE HEALING PIPELINE IS NOT BROKEN

The SauceDemo benchmark proves **every stage works correctly on HEAD**:

```
Stage 1: Execution → test-results.json produced (32s, exit=1)
Stage 2: ArtifactCollector → 1 artifact collected ✓
Stage 3: FailureClassifier → category=locator, healable=true ✓
Stage 4: DOM extraction → 2074 chars reconstructed ✓
Stage 5: Candidate generation → 4 candidates discovered ✓
Stage 6-8: Validation, apply, rerun → all passed ✓
```

The advisor pipeline runs correctly:
- FailureClassifier categorizes as `locator` (NOT `framework`)
- HealingStrategyRouter routes to `advisor` disposition (NOT `hard_stop`)
- DOM extractor, Rule Engine, and candidate ranking all execute
- Validation confirms the fix works

❌ The Regression is NOT in the Healing Logic

PDF symptoms:
- Old benchmark: ~14s full pipeline execution
- Current: ~1.1s exit with Framework/INCONCLUSIVE
- “Candidate generation never runs”
- “DOM extractor never executes”

Critical timing clue: 1.1s vs 32s proves the regression is **UPSTREAM** of the healing pipeline.

Root Cause Hypothesis

The INCONCLUSIVE gate exits early when:

```
artifacts.length === 0 && !runTrust.trustworthy
```

This happens when:
1. **Initial test execution fails at setup/framework level** (no test-results.json produced, or empty file)
2. **ArtifactCollector.collect fails** to parse the results file
3. **Results file is in wrong location** or has unexpected structure

The 1.1s timing suggests:
- ❌ NOT a full test run (that takes 30+ seconds)

- ✅ Likely a fast framework/setup failure (spec load error, import failure, missing config)
- ✅ Or results file not being found/parsed correctly

Instrumentation Added

Added detailed logging at each stage:

1. execution-result.ts (ArtifactCollector stage)

```
▶ STAGE: ArtifactCollector.collect
  - resultsFile path
  - resultsFileExists check
  - artifactCount
  - durationMs
```

2. failure-classifier.ts

```
▶ STAGE: FailureClassifier.classify
  - category, confidence, locator
  - recommendedStrategy
  - healableByLocatorSwap
```

3. healing-strategy-router.ts

```
▶ STAGE: HealingStrategyRouter.route
  - disposition (hard_stop vs advisor)
  - shouldAttemptLocatorHealing
  - rationale
```

4. server.ts (INCONCLUSIVE gate)

```
▶ STAGE: INCONCLUSIVE gate check
  - artifactCount
  - runTrustworthy
  - exitCode
  - Decision: PASSED or EXITING EARLY
```

Next Steps - Behavioral Bisect

Required Investigation (in order)

1. **Reproduce the 1.1s INCONCLUSIVE exit**
   - Run the specific test/environment that triggers the regression
   - Capture logs showing INCONCLUSIVE gate firing
   - Check: `artifactCount=0`, `runTrustworthy=false`, `exitCode!=0`

2. **Inspect the ArtifactCollector input**
   - Does test-results.json exist?
   - What's in the file? (empty? malformed JSON? wrong structure?)
   - Is the file path correct?

3. **Check the initial execution**
   - What is the actual exitCode?
   - Is stdout/stderr showing a framework error?
   - Are there spec-load errors (extractTopLevelErrors)?

4. **Compare execution contexts**
   - Benchmark uses: `ExecutionEngine.runAsync` directly
   - Production uses: ExecutionProvider → assembleExecutionResult
   - Are there differences in how results are collected?

Files to Check

- `src/core/execution/providers/local-execution-provider.ts` - How execution is invoked
- `src/core/execution/execution-result.ts` - How results file is located
- `src/core/artifact-collector.ts` - Parsing logic
- `src/core/execution-trust.ts` - Trust assessment logic

Commands to Run

```
# 1. Run with diagnostic logging to see exact failure point
DATABASE_URL=<url> node dist/api/server.js

# 2. Check if test-results.json is being created
ls -lah /path/to/repo/test-results.json

# 3. Inspect the results file structure
cat /path/to/repo/test-results.json | jq .

# 4. Run the specific failing test manually
cd /path/to/repo && npx playwright test <spec> --reporter=json
```

Key Insight from Benchmark

The advisor architecture works correctly:
- `framework` category → routes to `advisor` disposition ✓
- `unknown` category → routes to `advisor` disposition ✓
- Advisors (DOM, Rule, Pattern, AI) run and produce candidates ✓
- Only `assertion`, `navigation`, `api`, `environment` → hard_stop ✓

The healing strategy router is NOT blocking candidate generation.

Conclusion

The regression described in the PDF is **NOT** caused by:
- ✅ Healing strategy router
- ✅ Failure classifier
- ✅ Advisor pipeline
- ✅ Candidate generation logic
- ✅ Validation layer

The regression **IS** caused by:
- ❌ Initial test execution producing no artifacts

- ❌ OR artifact collection failing to parse results
- ❌ OR execution provider not finding results file

Next action: Reproduce the INCONCLUSIVE exit with diagnostic logs to see the exact artifact collection input/output.